

The Data Engineering Lifecycle In Depth

Lecture Notes for a 4-Hour Session - Course 1, Week 2

Based on materials by DeepLearning.AI and the textbook "Fundamentals of Data Engineering" by J

February 14, 2026

Contents

1	Part 1: The Data Engineering Lifecycle In Depth	2
1.1	Week 2 Overview	2
1.2	Data Generation in Source Systems	2
1.2.1	Common Source System Types	2
1.2.2	The Unpredictability of Source Systems	2
1.3	Ingestion	3
1.3.1	Batch vs. Streaming Ingestion	3
1.3.2	Other Ingestion Considerations	3
1.4	Storage	3
1.4.1	The Raw Ingredients of Storage	3
1.4.2	The Storage Hierarchy	4
2	Part 2: Queries, Transformation, and Serving	5
2.1	Queries, Modeling, and Transformation	5
2.1.1	Queries	5
2.1.2	Data Modeling	5
2.1.3	Transformation	5
2.2	Serving Data	6
2.2.1	Analytics	6
2.2.2	Machine Learning	6
2.2.3	Reverse ETL	6
3	Part 3: The Undercurrents of Data Engineering	7
3.1	Introduction to the Undercurrents	7
3.2	Security	7
3.2.1	The Principle of Least Privilege	7
3.2.2	Data Sensitivity	7
3.2.3	Cloud Security Considerations	7
3.2.4	Security as a Culture	7
3.3	Data Management	8
3.3.1	DMBOK Definition	8
3.3.2	Data Knowledge Areas	8
3.3.3	Data Governance and Data Quality	8
3.4	Data Architecture	8
3.4.1	Principles of Good Data Architecture	9
3.5	DataOps	9
3.5.1	Cultural Practices	9
3.5.2	Three Technical Pillars of DataOps	9
3.6	Orchestration	10
3.6.1	From Manual Execution to Orchestration	10
3.6.2	Directed Acyclic Graphs (DAGs)	10
3.7	Software Engineering	10
3.7.1	The Evolution of Coding in Data Engineering	10
3.7.2	Where You Will Write Code	11

4	Part 4: The Data Engineering Lifecycle on AWS	12
4.1	Lesson Introduction	12
4.2	The Lifecycle Stages on AWS	12
4.2.1	Source Systems	12
4.2.2	Ingestion	12
4.2.3	Storage	12
4.2.4	Transformation	12
4.2.5	Serving	13
4.3	The Undercurrents on AWS	13
4.3.1	Security	13
4.3.2	Data Management	13
4.3.3	DataOps	13
4.3.4	Orchestration	13
4.3.5	Data Architecture	13
4.3.6	Software Engineering	14
5	Week 2 Summary	14

1 Part 1: The Data Engineering Lifecycle In Depth

1.1 Week 2 Overview

This week dives deep into the data engineering lifecycle and undercurrents introduced in Week 1. The lifecycle begins with data generation from source systems, then moves through ingestion, transformation, storage, and serving of data for multiple use cases like analytics and machine learning. In simpler terms: get raw data from somewhere, turn it into something useful, and then make it available for downstream use cases.

The second lesson covers the undercurrents of the data engineering lifecycle: security, data management, DataOps, data architecture, orchestration, and software engineering. While the focus remains on building a high-level mental framework, the week concludes with how this framework comes together in practice on the AWS Cloud, including a hands-on end-to-end cloud data pipeline.

1.2 Data Generation in Source Systems

The first stage of the data engineering lifecycle is the generation of data in source systems. As a data engineer, you consume data from various sources but typically do not own or control them. These systems are created and maintained by other teams—software engineers from other internal departments, external vendors, or third-party platforms. Understanding how these systems work is essential because the data pipelines you build rely on the data generated from these sources.

1.2.1 Common Source System Types

Databases The most common source systems. These could be **relational databases**, represented as tables of related data, or other types of **NoSQL systems**, including key-value databases, document stores, and more. These databases might be part of the back end of a web or mobile application.

Files Data in the form of text files, audio files (e.g., MP3s), video, or other file types. While individual files might not seem like a “source system,” data engineers frequently need to download or be given access to files as part of their work.

APIs An **Application Programming Interface** allows you to issue a web request for data and receive it back in a certain format, such as XML or JSON.

Data Sharing Platforms Organizations may set up platforms to share data internally or with third parties.

IoT Devices An increasingly common source system. A **swarm of IoT devices** may stream data in real time (e.g., GPS trackers for delivery updates). This streaming data is typically sent to a database, and the source system owner might make it accessible via an API or data sharing platform.

1.2.2 The Unpredictability of Source Systems

In a perfect world, source systems would deliver data in a consistent and timely manner. In reality, source systems are unpredictable: they go down, schemas change without notice, or the data itself changes while the schema stays the same. As a data engineer, you will be more successful if you work directly with source system owners to understand:

- How their systems work and how they generate data
- How that data may be subject to change over time
- How those changes will impact the downstream systems you build

Developing good working relationships with the stakeholders of source systems is an underrated yet crucial part of successful data engineering.

1.3 Ingestion

Data ingestion means moving raw data from source systems into your data pipeline for further processing. Source systems and data ingestion represent the biggest bottlenecks of the data engineering lifecycle. One of the critical decisions when designing data ingestion processes is **how frequently** you need to ingest the data.

1.3.1 Batch vs. Streaming Ingestion

You can think of data being produced as a continuous series of events—clicks on a website, sensor measurements, or something else. In a sense, nearly all data is streamed at its source.

Batch Ingestion A convenient way to process the stream of events in large chunks, either on a predetermined time interval or as data reaches a preset size threshold. For example, you could handle an entire day’s worth of data in one single batch. Batch processing has long been the default way to ingest data and remains a practical and popular approach, particularly for analytics and machine learning use cases.

Streaming Ingestion Ingesting and providing data to downstream systems in a continuous, near real-time fashion—possibly less than one second after production. This requires specific tools such as an **event streaming platform** or a **message queue** to continuously ingest streams of events.

Before committing to streaming ingestion, ask yourself:

- What actions can you take on real-time data that would be an improvement over batch data?
- Would streaming ingestion cost more than batch in terms of time, money, maintenance, and potential downtime?
- How will choosing stream ingestion over batch (or vice versa) influence the rest of your data pipeline?

Adopt streaming only after identifying a business use case that justifies the trade-offs. Streaming ingestion generally coexists with batch processing; rarely do data engineers build a purely streaming pipeline with no batch components. Instead, you choose where the boundaries between batch and streaming will occur.

1.3.2 Other Ingestion Considerations

Change Data Capture (CDC) A technique to trigger certain ingestion processes based on data changes in the source system, rather than ingesting on a fixed schedule.

Push vs. Pull Whether a source system **pushes** data to you or you actively **pull** it from the source.

1.4 Storage

In your work as a data engineer, the function, performance, and limitations of the systems you build will have a lot to do with the storage solutions you choose. Storage is not a single step but a continuous process; data is stored after ingestion, between transformations, and before being served.

1.4.1 The Raw Ingredients of Storage

Magnetic Disk Still forms the backbone of modern storage systems, primarily because of low cost. At current pricing, disk storage is about 2–3 times cheaper than solid-state storage.

Solid-State Drives (SSDs) Flash memory offering faster read/write speeds than magnetic disks, commonly found in laptops and smartphones.

RAM (Memory) Offers much faster read and write speeds than SSDs or disks, making it a critical component of many architectures. However, RAM storage could be 30–50 times more expensive than solid-state storage and is **volatile**—if the system loses power, memory is typically lost instantly.

Other Raw Ingredients In modern cloud architectures, networking, CPU, serialization, compression, and caching are all critical components for storing data in distributed systems across multiple clusters and data centers.

1.4.2 The Storage Hierarchy

If you think about storage as a hierarchy, it can be arranged in three tiers:

1. **Raw Ingredients:** The physical components (disk, RAM, SSD) and non-physical components (networking, serialization, compression, caching).
2. **Storage Systems:** Built from raw ingredients. These include database management systems, object storage platforms (e.g., Amazon S3), Apache Iceberg, Hudi, cache and memory-based storage systems, and streaming storage.
3. **Storage Abstractions:** Combinations of storage systems that allow you to achieve high-level data storage needs without worrying about low-level details. Examples include the **data warehouse**, the **data lake**, and the more recent **data lakehouse**.

As a data engineer, you may spend much of your time operating at or near the top of this hierarchy. However, you will be most effective if you take the time to understand the inner workings, capabilities, and limitations of your entire storage solution right down to the raw ingredients. Many practicing data engineers do not deeply understand their storage systems, which leads to unfortunate consequences in performance and cost.

2 Part 2: Queries, Transformation, and Serving

2.1 Queries, Modeling, and Transformation

The transformation stage is where you, as a data engineer, start to add value. Ingesting and storing raw data from source systems does not directly add value for downstream stakeholders—transformation is the “turn it into something useful” stage. In reality, this stage is made up of three parts: **queries**, **modeling**, and **transformation**.

2.1.1 Queries

When you query data, you issue a request to read records from a database or other storage system. While there are many query languages, these courses focus on **Structured Query Language (SQL)**, which remains a popular and universal query language. A query may involve cleaning, joining, and aggregating data across many datasets, as well as filtering with SQL expressions.

There is more than one way to write a query, and **poorly written queries** can have negative consequences:

- Impacting the performance of a source database
- Causing a **row explosion**, where a join between tables produces many more records than anticipated, potentially overwhelming storage infrastructure
- Causing downstream delays in reporting or analytics

Most data engineers can read and write SQL but are unfamiliar with how queries work under the hood, which can have unforeseen consequences in the architectures they build.

2.1.2 Data Modeling

A **data model** represents the way data relates to the real world. Data modeling involves deliberately choosing a coherent structure for your data and is a crucial step in making data useful for the business.

Normalization Data from upstream relational source databases is often **normalized**—stored in separate tables (e.g., product information, order details, customer information) with complex relationships between them.

Denormalization To serve an analyst’s needs, you might need to **denormalize** the data, modeling it in a way that allows quick and efficient querying for reports.

A good data model reflects your organization’s processes, definitions, workflows, and logic. Success with data modeling requires working with stakeholders to understand their terminology (e.g., what “customer” means to different departments) and business goals.

2.1.3 Transformation

Data must be manipulated, enhanced, and saved for downstream use. Transformation occurs multiple times throughout the data engineering lifecycle:

1. **At the source:** A timestamp may be added to a record while it is still on a source system.
2. **In-flight during ingestion:** Transformations applied while data is being ingested.
3. **Immediately after ingestion:** Basic transformations to map data to correct types and put records into standardized formats.
4. **Within a streaming pipeline:** Enriching a record with additional fields and calculations before it is sent to a data warehouse.
5. **Further downstream:** Schema transformation, denormalization, large-scale aggregation for reporting, or featurization for machine learning model training.

2.2 Serving Data

Serving is the final stage of the data engineering lifecycle. It is about more than just making data available—it is when stakeholders extract business value from data. Data has value when it is used for practical end use cases.

2.2.1 Analytics

Analytics is the process of identifying key insights and patterns within data. As a data engineer, you are almost guaranteed to serve data that powers one or more of the three most common forms of analytics:

Business Intelligence (BI) Analysts explore historical and current business data to discover insights. Data is presented in the form of reports or dashboards that help users make strategic and actionable decisions. For example, sales and marketing teams use BI reports to spot patterns, monitor campaign engagement, regional sales, and customer experience metrics.

Operational Analytics Monitoring real-time data for immediate action. For example, an e-commerce platform team might monitor a dashboard with real-time performance metrics for their website, needing to know immediately if the website goes down so they can triage the situation. The data engineer’s role is to ingest, transform, and serve event data from application logs to populate such dashboards.

Embedded Analytics External or customer-facing analytics—a newer trend. Examples include banking dashboards showing spending trends across categories, or smart thermostat mobile applications showing current and historical temperature data. As a data engineer, your job is serving real-time and historical data for use in user-facing applications.

2.2.2 Machine Learning

With the rise of machine learning, your role as a data engineer will likely involve serving data for ML use cases. This can involve additional complexities:

- Serving **feature stores** of data that facilitate model training
- Serving data for **real-time inference**
- Supporting **metadata and cataloging systems** that track data history and lineage

2.2.3 Reverse ETL

With reverse ETL, you take transformed data, analytics, and perhaps ML model output and **feed it back into source systems**. For example:

1. Ingest data from a CRM system (names, contact info, form data, client information).
2. Transform the data and store it in a data warehouse.
3. Analysts retrieve the data to train a **lead scoring model** that identifies the most promising clients.
4. The model results are returned to the data warehouse and pushed back into the CRM as an enhancement of the client data already stored there.

The name “reverse ETL” is not so much a description of what is happening as it is a label for a lack of a better name. This practice is increasingly common and likely to be part of your role as a data engineer.

3 Part 3: The Undercurrents of Data Engineering

3.1 Introduction to the Undercurrents

As the field of data engineering matures, the continued abstraction and simplification of tools has expanded the scope of the role beyond just tools and technologies. Modern data engineering incorporates traditional enterprise practices such as data management and cost optimization, as well as newer practices like DataOps. The **undercurrents** are a set of practices that apply across the entire lifecycle:

- Security
- Data Management
- DataOps
- Data Architecture
- Orchestration
- Software Engineering

3.2 Security

As a data engineer, you are entrusted with sensitive data—personal and private information of clients or proprietary business information. Security is about bringing together the right set of principles, protocols, and cultural practices to ensure security in the systems you build.

3.2.1 The Principle of Least Privilege

When managing a data pipeline and serving data to end users, you give access to data and system resources to other users or applications. The **Principle of Least Privilege** states:

Give users or applications access to only the essential data and resources they need to do their jobs, and only for the duration that is required.

This principle applies not only to others in your organization but also to yourself: do not operate from the root shell when it is not necessary, and do not use administrator or super-user permissions unless absolutely essential.

3.2.2 Data Sensitivity

Beyond access control, think about data sensitivity. The best way to protect sensitive data is **not to ingest it** into your system in the first place. If you do not have a clear purpose for ingesting and storing sensitive data, simply do not do it, and you will have entirely removed the risk of accidentally leaking that data.

3.2.3 Cloud Security Considerations

In today's cloud-centric world, security requires understanding:

- Identity and Access Management (IAM) roles and policies
- Encryption methods
- Networking protocols and configurations

3.2.4 Security as a Culture

Security is not just about principles and protocols—it is also about people. Adopt a **defensive mindset**: be cautious when asked for credentials or sensitive data, imagine potential attack and leak scenarios, and design your data pipelines with them in mind. Many of the biggest security breaches have been caused by individuals ignoring basic precautions (sharing passwords insecurely, falling victim to phishing attacks, or accidentally leaving an S3 bucket or database exposed to the public Internet). Adhering to the *letter* of security without a culture and spirit of security is what the textbook calls **security theater**. True security emerges from a culture where every team member recognizes their role in protecting data.

3.3 Data Management

Data can be an incredibly valuable business asset, but only when it is managed properly. The **Data Management Association International (DAMA)** provides the **Data Management Body of Knowledge (DMBOK)**, a comprehensive reference for data management practices.

3.3.1 DMBOK Definition

The DMBOK defines data management as:

The development, execution, and supervision of plans, programs, and practices that deliver, control, protect, and enhance the value of data and information assets throughout their lifecycles.

3.3.2 Data Knowledge Areas

The DMBOK breaks down data management into 11 knowledge areas, including:

- Data Governance
- Data Modeling
- Data Integration and Interoperability
- Metadata Management
- Data Security

Data governance touches all other areas of data management, and many knowledge areas interact with each other through data governance practices.

3.3.3 Data Governance and Data Quality

Data governance is a data management function to ensure the quality, integrity, security, and usability of data collected by an organization. A key concern within governance is **data quality**:

- **High-quality data** is accurate, complete, discoverable, and available in a timely manner. It represents exactly what stakeholders expect in terms of well-defined schema and data definitions.
- **Low-quality data** might be inaccurate, incomplete, or otherwise unusable. It can cause stakeholders to waste time, make poor decisions, or lose trust in the data team entirely.

3.4 Data Architecture

Data architecture is a roadmap or blueprint for your data systems. In our book *Fundamentals of Data Engineering*, data architecture is defined as:

The design of systems to support the evolving data needs of an enterprise, achieved by flexible and reversible decisions reached through a careful evaluation of trade-offs.

Key aspects of this definition:

- A good design supports the data needs of **today and tomorrow**—architecture is an ongoing effort.
- Good design is achieved through **flexible and reversible decisions**, making it easier to evolve the architecture as needs change.
- All decisions involve a careful **evaluation of trade-offs** in performance, cost, scalability, and other parameters.

Cloud computing makes flexible and reversible decisions far easier than the on-premises era, where millions of dollars in server hardware could lock you into a system for years.

3.4.1 Principles of Good Data Architecture

1. **Choose common components wisely.** Common components are used by multiple teams across the organization. A good choice provides the right features for individual projects while facilitating collaboration.
2. **Plan for failure.** Design not only for when everything works as expected, but also for when things break.
3. **Architect for scalability.** Scalable systems scale up to meet demand and scale down to minimize costs when demand recedes.
4. **Architecture is leadership.** Thinking like an architect helps you lead and mentor other team members as your skills develop.
5. **Always be architecting.** Architecture design is not a one-time effort—constantly evaluate your systems against the evolving needs of your organization.
6. **Build loosely coupled systems.** A loosely coupled system is built from individual components that can be easily swapped out without re-architecting the whole system.
7. **Make reversible decisions.** Choosing easily interchangeable components means you can reverse prior decisions and swap out components as needs evolve.
8. **Prioritize security.** Security principles like Least Privilege and Zero Trust are central to your role.
9. **Embrace FinOps.** FinOps brings together business priorities of finance and DataOps. On the cloud, most data systems are pay-as-you-go and readily scalable; embracing FinOps lets you design systems optimized for both cost and potential revenue generation.

3.5 DataOps

Around 2007, **DevOps** emerged in software development to break silos between development teams (who write and test code) and deployment teams (who deploy and maintain code). DevOps borrows from lean and agile methodologies to accomplish removal of bottlenecks, reduction of waste, quick identification of problems, and rapid iteration. **DataOps** applies these same principles to data products.

3.5.1 Cultural Practices

DataOps is first and foremost a set of cultural habits and practices, borrowed from agile methodology:

- Prioritizing communication and collaboration with business stakeholders
- Continuously learning from successes and failures
- Taking an approach of rapid iteration toward improvements

3.5.2 Three Technical Pillars of DataOps

Automation One of the DevOps practices that accelerated the software build lifecycle is **Continuous Integration and Continuous Delivery (CI/CD)**. DataOps employs a similar automation framework: in addition to managing changes in code, configuration, and environment, DataOps focuses on change management for data processing pipelines and the data itself.

The progression of automation in a data pipeline typically follows three stages:

1. **Manual execution:** Manually triggering each step—useful for prototyping but prone to errors and inefficient in the long run.
2. **Pure scheduling:** Setting each task to start at a particular time of day. This can fail when ingestion tasks take longer than expected or when upstream tasks fail silently.
3. **Orchestration:** Using frameworks like Apache Airflow to check dependencies between tasks, automatically start subsequent tasks once previous ones complete, and notify you of errors.

Observability and Monitoring Any data pipeline is bound to fail eventually. As Werner Vogels, CTO of Amazon Web Services, says: “Everything fails all the time.” Without close observation and monitoring, you may only discover failures when downstream stakeholders find problems in their reports or dashboards. Bad data can linger in reports for months or even years due to undiscovered failures.

Incident Response Using observability and monitoring capabilities to rapidly identify root causes of an incident and resolve it as quickly and reliably as possible. Incident response involves not just technology and tools but also **open and blameless communication** and coordination among team members.

3.6 Orchestration

Like an orchestra conductor coordinating instruments and voices, a data engineer coordinates and manages the tasks in data pipelines. Orchestration is both a central component of DataOps and a separate undercurrent spanning the entire lifecycle.

3.6.1 From Manual Execution to Orchestration

Manual Execution Manually triggering each step. Useful for prototyping but not sustainable for production data processing.

Pure Scheduling Tasks run automatically at particular times or frequencies. Problems arise when scheduled tasks fail or take longer than expected, causing incomplete or stale data to propagate through the pipeline.

Orchestration Frameworks Open-source frameworks like **Apache Airflow**, **Dagster**, **Prefect**, and **Mage** allow you to automate data pipelines with complex dependencies and monitoring capabilities. Features include:

- **Dependency-based task triggering** (verify previous tasks completed before starting the next)
- **Event-driven triggers** (e.g., start ingestion when new data is available)
- **Monitoring and alerting** (notify when tasks fail or exceed time limits)

3.6.2 Directed Acyclic Graphs (DAGs)

Many orchestration frameworks require you to set up your data pipeline as a **Directed Acyclic Graph (DAG)**:

- **Directed:** The flow of data goes only in one direction.
- **Acyclic:** There are no loops—data does not flow back to a previous step.
- **Graph:** Composed of nodes (tasks) and edges (data flow between tasks).

You can think of a DAG as a flow chart for how data moves through your pipelines. Within your orchestration framework, you specify criteria and dependencies for how each task should be triggered and what monitoring and alerts to set up.

3.7 Software Engineering

As a data engineer, you need to know how to read and write code—not just hacking together code that works right now, but writing **production-grade code** that is clean, readable, testable, and deployable.

3.7.1 The Evolution of Coding in Data Engineering

In the not-too-distant past, there was no official “data engineering” profession—just software engineers who occasionally dealt with data. As businesses recognized the value of data, the data-oriented component of software engineering became more intensive and eventually emerged as its own field. Today, existing services and applications allow data engineers to write much less code than their predecessors. However, the code you do write must be of top quality.

3.7.2 Where You Will Write Code

- **Core data processing code** at all stages of the lifecycle (ingestion, transformation, serving), using frameworks and languages such as SQL, Spark, or Kafka.
- **Programming languages:** Python, Java/Scala (JVM languages), Bash for command-line operations, and potentially Rust or Go.
- **Open-source framework development:** Adopting and extending open-source frameworks for specific use cases, contributing back via pull requests.
- **Infrastructure as Code (IaC)** and **Pipeline as Code** solutions.
- **General-purpose problem solving** at all stages of the lifecycle.

Building strong foundational software engineering skills ensures you can move between languages and frameworks with minimal difficulty. Making friends with software engineers in your organization and learning from them how to write great code is well worth your time.

4 Part 4: The Data Engineering Lifecycle on AWS

4.1 Lesson Introduction

Now that you are familiar with the data engineering lifecycle and undercurrents, it is time to see how these concepts translate to tools and technologies on the AWS cloud. While other cloud providers exist (Microsoft Azure, Google Cloud Platform), all the concepts from this course are relevant regardless of platform—only some tooling and implementation details differ. AWS is currently a leading cloud provider, and the course partners with them for hands-on labs.

4.2 The Lifecycle Stages on AWS

4.2.1 Source Systems

Amazon RDS Relational Database Service provisions database instances with the relational database engine of your choice (e.g., MySQL, PostgreSQL). RDS simplifies operational overhead for provisioning, hosting, patching, and upgrading.

Amazon DynamoDB A serverless NoSQL database option. Tables are virtually unlimited in size. DynamoDB has a flexible schema and is suited for applications requiring low-latency access to large volumes of data (gaming, IoT, mobile apps, real-time analytics).

Amazon Kinesis Data Streams For streaming sources, enables real-time streaming of user activities (e.g., from a sales platform).

Amazon SQS Simple Queue Service for handling messages in your data pipelines.

Amazon MSK Managed Streaming for Apache Kafka, making it easier to run Kafka workloads on AWS with managed underlying infrastructure.

4.2.2 Ingestion

AWS DMS Database Migration Service migrates and replicates data from a source to a target in an automated way, useful when ingesting from databases.

AWS Glue ETL Offers features that support data integration processes. Used in most lab exercises for ingestion.

Amazon Kinesis Data Streams & Amazon Data Firehose For ingesting data from streaming sources.

4.2.3 Storage

Amazon Redshift A traditional cloud data warehouse option for structured analytical data.

Amazon S3 Simple Storage Service for object storage, serving as the backbone of a data lake.

Lakehouse Architecture Combining services (e.g., Redshift and S3) for seamless access to both structured data in a data warehouse and unstructured data in an object storage data lake.

4.2.4 Transformation

AWS Glue Primary transformation service used in the courses.

Apache Spark Can be used in combination with Glue or as an alternative.

dbt (data build tool) Another transformation tool that can be used in combination with Glue or as an alternative, depending on needs.

4.2.5 Serving

Analytics Use Case • **Amazon Athena** or **Redshift** for querying structured and unstructured data

- **Jupyter Notebooks** for dashboards and analysis
- **Amazon QuickSight** for managed dashboards
- **Apache Superset** and **Metabase** as open-source dashboard options

AI/Machine Learning Use Case • Serving batch data for model training

- Vector database options for product recommenders and use with large language models

4.3 The Undercurrents on AWS

4.3.1 Security

Shared Responsibility Model AWS is responsible for security of the data centers and services they provide. You are responsible for the security of the systems you build with those resources (e.g., setting correct access permissions on S3 buckets).

IAM (Identity and Access Management) Set up roles and permissions to control access to AWS resources. **IAM roles** give users or applications access to temporary credentials that automatically rotate and provide appropriate AWS API permissions.

Network Security Services and features like **Amazon VPC** (Virtual Private Cloud) and **security groups** (instance-level firewalls) are key aspects of implementing secure data pipelines.

4.3.2 Data Management

AWS Glue Crawlers & Glue Data Catalog Discover, create, and manage metadata for data stored in Amazon S3 or other storage and database systems.

AWS Lake Formation Helps centrally manage and scale fine-grained data access permissions, related to both data management and security (data privacy and discovery).

4.3.3 DataOps

Amazon CloudWatch Collects metrics and provides monitoring features for cloud resources, applications, and even on-premises resources.

Amazon CloudWatch Logs Stores and analyzes operational logs.

Amazon SNS Simple Notification Service provides a means of setting up notifications (between applications or via text/email) triggered by events within your system.

Open-Source Options Tools like Monte Carlo or BigEye for observability.

4.3.4 Orchestration

Apache Airflow The industry-standard orchestration tool. Can be implemented as open source or used as a managed version from AWS (Amazon MWAA—Managed Workflows for Apache Airflow).

Newer Alternatives Dagster, Prefect, and Mage aim to solve some issues that Airflow is not designed for.

4.3.5 Data Architecture

AWS Well-Architected Framework A set of principles and practices developed by AWS to help build systems with an eye towards operational efficiency, security, scalability, and sustainability.

4.3.6 Software Engineering

Amazon CodeDeploy Automates code deployment.

CI/CD Tools Various options for continuous integration and delivery.

Git and GitHub For version control throughout the data engineering workflow.

5 Week 2 Summary

This week provided a deep dive into every stage of the data engineering lifecycle and each of the six undercurrents. The key takeaways are:

1. **Understand your source systems:** Work directly with source system owners and build good relationships to anticipate changes and avoid pipeline failures.
2. **Choose ingestion patterns wisely:** Adopt streaming ingestion only when a clear business use case justifies the trade-offs over batch processing.
3. **Be storage savvy:** Understand the full storage hierarchy—from raw ingredients to storage abstractions—to optimize for performance and cost.
4. **Add value through transformation:** Queries, modeling, and transformation are where raw data becomes useful for analytics, ML, and reverse ETL.
5. **Embrace the undercurrents:** Security, data management, DataOps, architecture, orchestration, and software engineering are cross-cutting concerns that determine the quality and reliability of everything you build.
6. **Map concepts to tools:** Understanding how lifecycle stages and undercurrents map to specific AWS services bridges the gap between theory and practice.